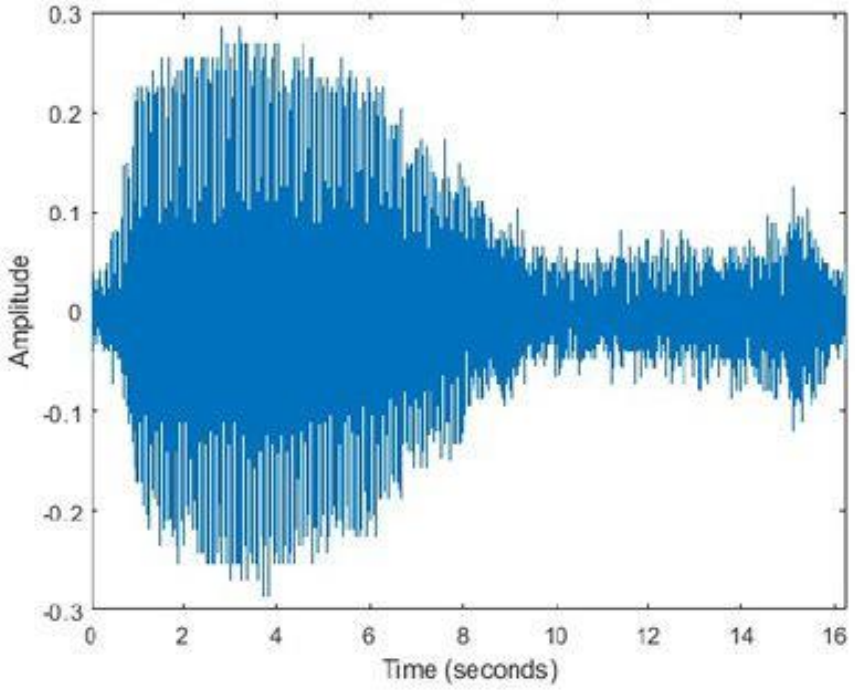
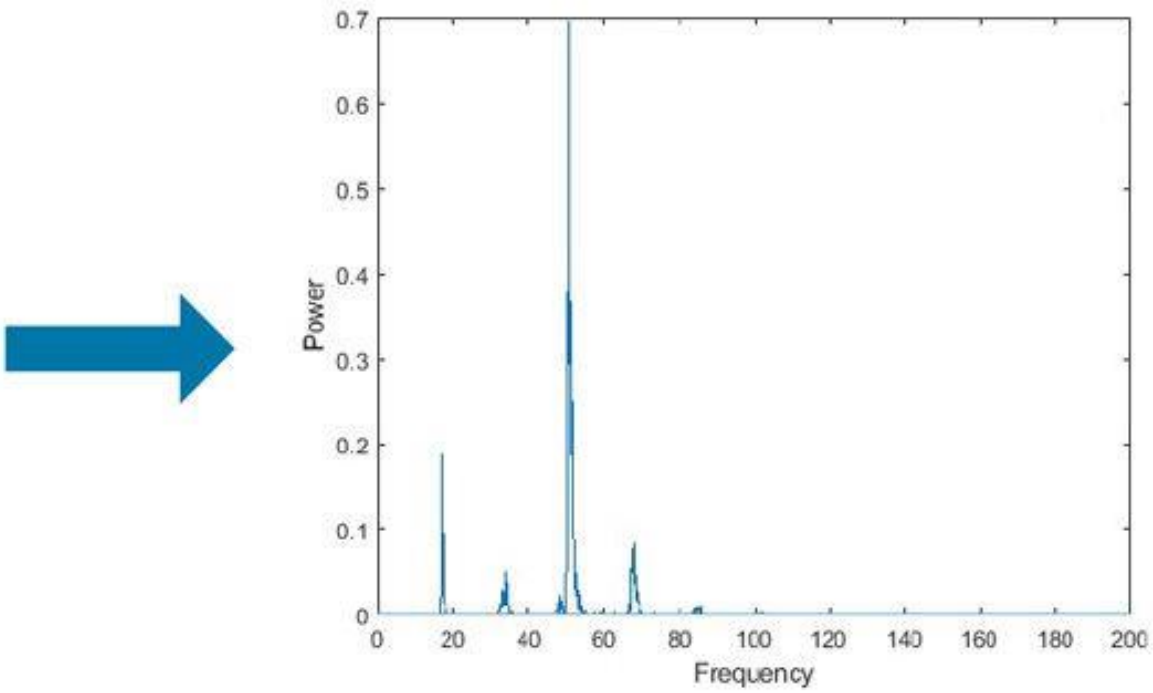
Fast Fourier transform on GPU

Torge Schwark

Fast Fourier transform (FFT)

- Transforms a signal from the **time domain** into the **frequency domain**
- Reveals which **frequencies** are present in the signal
- Computes the **amplitude (magnitude)** of each frequency component





Computing a FFT

An FFT of size N is composed of two FFTs of size $N/2$:

$$\mathrm{FFT}_N(x) = \mathrm{FFT}_{N/2}(x_{\text{even}}) \oplus \mathrm{FFT}_{N/2}(x_{\text{odd}})$$

Combination step (Butterfly):

$$X_k = E_k + W_N^k \cdot O_k$$

$$X_{k+N/2} = E_k - W_N^k \cdot O_k$$

Interpretation:

- FFT_N is built from two smaller FFTs: $\mathrm{FFT}_{N/2}$
- $x_{\text{even}} = (x_0, x_2, x_4, \dots)$
- $x_{\text{odd}} = (x_1, x_3, x_5, \dots)$
- $E_k = \mathrm{FFT}_{N/2}(x_{\text{even}})$
- $O_k = \mathrm{FFT}_{N/2}(x_{\text{odd}})$

How Bit-Reversal Order Emerges from Recursive FFT

We consider $N = 8$ with indices:

$$[0, 1, 2, 3, 4, 5, 6, 7]$$

Step 1: First Split (Even / Odd)

$$\text{Even indices: } [0, 2, 4, 6]$$

$$\text{Odd indices: } [1, 3, 5, 7]$$

Step 2: Recursive Split

Split each group again:

$$[0, 2, 4, 6] \to [0, 4] \mid [2, 6]$$

$$[1, 3, 5, 7] \to [1, 5] \mid [3, 7]$$

Step 3: Final Split

$$[0, 4] \to [0] \mid [4]$$

$$[2, 6] \to [2] \mid [6]$$

$$[1, 5] \to [1] \mid [5]$$

$$[3, 7] \to [3] \mid [7]$$

Final Order (Read Top to Bottom)

$$[0, 4, 2, 6, 1, 5, 3, 7]$$

Bit Reversal

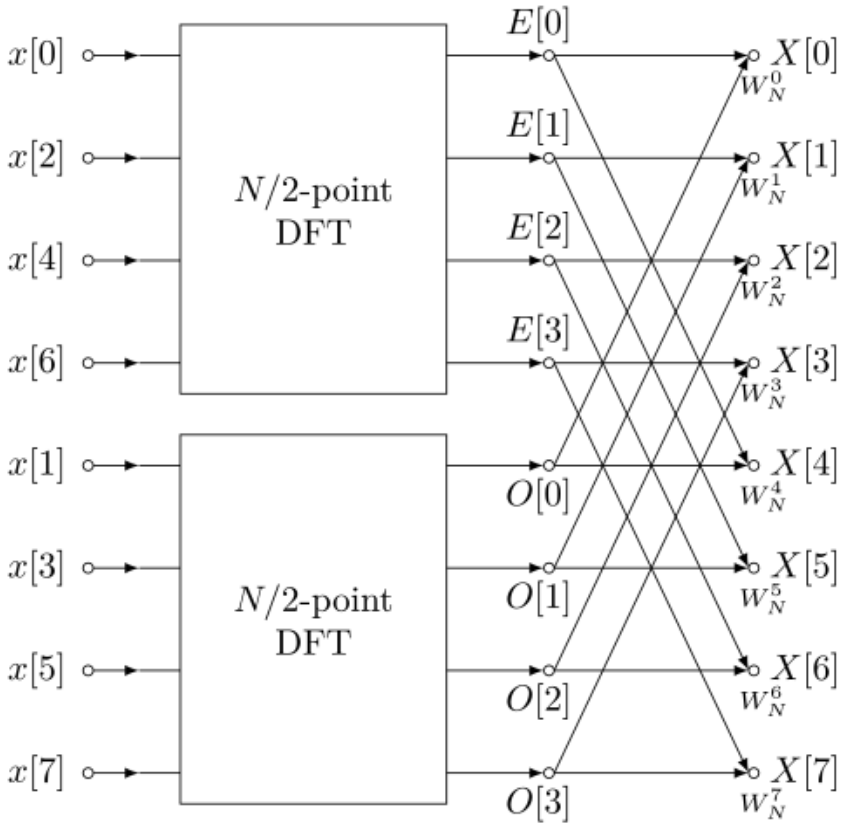
Programming FFT (Cooley–Tuk)

Iterative

Bit-Reversal Example for $N = 8$

i (decimal)	binary	reversed	j (decimal)
0	000	000	0
1	001	100	4
2	010	010	2
3	011	110	6
4	100	001	1
5	101	101	5
6	110	011	3
7	111	111	7

Rekursive



Code

```cpp
bit_reverse_kernel_base<<<blocks, threads>>>(d_x, N, logN);
CUDA_CHECK(cudaDeviceSynchronize());
auto start = Clock::now();

int stage = 0;

for (int m = 2; m <= N; m <<= 1)
{
    int half = m >> 1;

    int butterflies = N >> 1;

    int stageBlocks =
        (butterflies + threads - 1) / threads;

    // compute base twiddle
    double angle = -2.0 * M_PI / m;

    cuFloatComplex base_twiddle =
        make_cuFloatComplex(cos(angle), sin(angle));

    fft_stage_kernel_base<<<stageBlocks, threads>>>(
        d_x,
        base_twiddle,
        N,
        m);
```

```cpp
__global__
void bit_reverse_kernel_base(cuFloatComplex* x, int N, int logN)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= N) return;

    int j = reverse_bits(i, logN);

    if (i < j)
    {
        cuFloatComplex tmp = x[i];
        x[i] = x[j];
        x[j] = tmp;
    }
}
```

Log(N) times

```cpp
__global__
void fft_stage_kernel_base(
    cuFloatComplex* __restrict__ x,
    cuFloatComplex base_twiddle,
    int N,
    int m)
{
    int tid = blockIdx.x * blockDim.x + threadIdx.x;

    int half = m >> 1;

    int butterflies = N >> 1;

    if (tid >= butterflies) return;

    int group = tid / half;
    int j     = tid % half;

    int base = group * m;

    int i1 = base + j;
    int i2 = i1 + half;

    cuFloatComplex u = x[i1];
    cuFloatComplex v = x[i2];

    // compute twiddle
    cuFloatComplex w = cpow_int(base_twiddle, j);

    cuFloatComplex t = cmul(w, v);

    x[i1] = cuCaddf(u, t);
    x[i2] = cuCsubf(u, t);
}
```

Results

N	seque (ms)	CUDA (ms)	FFTW (ms)	CUDA TWIDLE(MS)	Seq x	CUDA x	CUDA TWIDLE x
256	0.0024	0.1460	0.0004	0.0996	6.6819	398.7245	272.0194
512	0.0052	0.1877	0.0011	0.1137	4.7178	171.3306	103.7977
1024	0.0111	0.2807	0.0014	0.1519	8.1712	205.7670	111.3657
2048	0.0254	0.2469	0.0026	0.0906	9.6154	93.4632	34.3028
4096	0.0528	0.2386	0.0069	0.0961	7.5942	34.3424	13.8344
8192	0.1205	0.2070	0.0192	0.1321	6.2822	10.7903	6.8842
16384	0.2893	0.2442	0.0349	0.1325	8.2789	6.9880	3.7921
32768	0.5905	0.2090	0.0678	0.1533	8.7126	3.0844	2.2621
65536	1.2550	0.3071	0.1401	0.1366	8.9606	2.1928	0.9756
131072	2.7092	0.3843	0.3386	0.1450	8.0015	1.1352	0.4281
262144	6.1661	0.5669	1.2737	0.1936	4.8412	0.4451	0.1520
524288	14.0811	0.8469	4.3580	0.3212	3.2311	0.1943	0.0737
1048576	31.2201	1.6544	13.4038	0.8033	2.3292	0.1234	0.0599
2097152	70.4876	3.1571	42.4715	1.6187	1.6596	0.0743	0.0381
4194304	176.9316	6.3220	128.6205	3.2946	1.3756	0.0492	0.0256
8388608	386.1293	28.7419	292.5958	18.8900	1.3197	0.0982	0.0646
16777216	857.3803	29.2618	623.5231	35.7050	1.3751	0.0469	0.0573

Up to 50 Times Faster than Library CPU FFT implementation!